Understanding MHA, MQA, GQA & MLA

From Multi-Head Attention to Latent KV Compression

A technical handbook on how query, key, and value head structure changes the KV cache, decode bandwidth, model capacity, and modern LLM inference - from standard multi-head attention through multi-query and grouped-query attention to DeepSeek's Multi-head Latent Attention.

@techNmak

How to read this handbook

MHA, MQA, GQA, and MLA are often presented as four boxes in a diagram. That view is useful, but it hides the reason these designs exist. The important questions are not just *how many heads are drawn*. They are:

1. **How many query heads produce distinct attention distributions?**
2. **How many distinct key/value representations must be stored for every previous token?**
3. **What representation is actually cached during autoregressive decoding?**

For MHA, MQA, and GQA, the cleanest mental model is to keep the number of query heads separate from the number of key/value heads. For MLA, that model stops being sufficient: the main saving comes from caching a low-dimensional latent representation rather than ordinary full per-head keys and values.[1][2][3][4]

> **Central distinction.** MHA, GQA, and MQA can be described primarily by *KV-head multiplicity*. MLA changes the *cached representation itself*. Calling MLA "GQA with even fewer heads" is therefore a false mental model.

This handbook focuses on decoder-style autoregressive inference because that is where persistent KV-cache capacity and repeated cache reads become especially important. Training behavior is discussed separately where the distinction matters.

Notation

We use L for the number of attention layers, T for cached sequence length, b for bytes per cached scalar, H_Q for query-head count, H_{KV} for key/value-head count, and d_h for the ordinary per-head key/value width unless stated otherwise.

For MHA/MQA/GQA with equal key and value widths, the cache payload for one sequence is

$$M_{\mathrm{KV}} = 2LTH_{KV}d_h b.$$

The factor of two is for keys and values. This is a capacity formula, not a universal statement about physical HBM traffic.

1. Start with one attention head

For one causal self-attention head at decode step t, the new query $\boldsymbol{q}_t$ is compared with keys from positions $1, \ldots, t$. If $\boldsymbol{k}_j, \boldsymbol{v}_j \in \mathbb{R}^{d_h}$, the output is

$$\boldsymbol{o}_t = \sum_{j=1}^{t} p_{tj} \boldsymbol{v}_j, \qquad p_{tj} = \mathrm{softmax}_j\left(\frac{\boldsymbol{q}_t^\top \boldsymbol{k}_j}{\sqrt{d_h}}\right).$$

The important inference detail is temporal asymmetry. The query for the new token is used once. The keys and values for previous tokens will be needed again when later tokens are generated. That is why decoder inference stores previous K/V tensors in a KV cache.

Without caching, a decoder would repeatedly recompute keys and values for the whole prefix. With caching, the model computes the new token's K/V once, appends them to the cache, and reuses the stored history on subsequent steps.

> The KV cache is not an optional "speed trick" added outside attention. It is the standard way to avoid recomputing past key/value projections during autoregressive decoding. Its size therefore becomes an architectural concern when contexts and batches grow.

The next question is what happens when attention has many heads.

2. Why multiple heads exist

The original Transformer applies multiple attention heads in parallel. Each head receives its own projected queries, keys, and values, computes an attention result, and the head outputs are concatenated and projected back to the model space.[1]

In the common equal-width case,

$$\boldsymbol{q}_i = W_i^Q \boldsymbol{h}, \qquad \boldsymbol{k}_i = W_i^K \boldsymbol{h}, \qquad \boldsymbol{v}_i = W_i^V \boldsymbol{h},$$

for head $i \in \{1, \ldots, H\}$. Each head can learn a different query-key compatibility function and a different value projection.

The usual explanation that "different heads attend to different things" is intuitive but incomplete. Architecturally, separate K/V heads also mean separate K/V projection parameters and separate cached K/V vectors during decoding.

This second consequence is what motivates MQA and GQA. They do not primarily ask whether multiple query heads are useful. They ask whether every query head must own a distinct key and value representation for every token in the prefix.

> **Do not conflate head count with model width.** In many architectures, $d_{\text{model}} = H d_h$. That is a common design choice, not a mathematical law. DeepSeek-V2's MLA configuration, for example, uses 128 attention heads of width 128 while the hidden dimension is 5120.[4]

3. Multi-Head Attention (MHA)

In standard MHA, query, key, and value head counts match:

$$H_Q = H_{KV} = H.$$

For every query head i, there is a corresponding key head and value head. During decoding, every previous token therefore contributes H key vectors and H value vectors to each attention layer's cache.

A compact per-token cache count across all L layers is

$$N_{\text{MHA}} = 2LHd_h$$

scalars. With storage width b bytes,

$$M_{\text{MHA}} = 2LTHd_hb.$$

This linear dependence on T is easy to underestimate. The cache grows with sequence length, batch size, layer count, KV-head count, head width, and storage precision. The model weights may fit comfortably while the cache still constrains the number of concurrent long-context requests.

MHA remains an important reference point because it preserves independent K/V projections for every query head. But that representational richness is exactly what makes its cache large.

4. Training and decoding stress different resources

During full-sequence training, all token positions are processed in parallel. There is no long-lived serving cache that must survive across one-token decode iterations. Attention still consumes activation memory and bandwidth, but the persistent KV-cache problem is specifically an inference concern.

During autoregressive decoding, each step creates only one new query position per sequence, while the model repeatedly attends over an ever-growing prefix. The previously computed K/V tensors must be available for every step. Shazeer's MQA paper identified the repeated loading of these large K/V tensors as a major memory-bandwidth cost of incremental decoding.[2]

This is why MQA/GQA are best understood as inference-oriented architectural changes. They can also change training parameter counts and projection work, but their signature benefit is that fewer distinct K/V vectors are produced and cached.

> **Decode mental model.** Query work is mostly "new token now." KV storage is "all previous tokens, every layer, every active sequence." Reducing the persistent K/V representation attacks the part that accumulates with context length and batch size.

5. The KV-cache formula, derived

For one token at one layer, suppose there are H_{KV} key heads and H_{KV} value heads, each of width d_h.

Keys contribute $H_{KV}d_h$ scalars. Values contribute another $H_{KV}d_h$. Therefore:

$$\text{scalars per token per layer} = 2H_{KV}d_h.$$

Across L layers and T cached token positions:

$$N_{\text{KV}} = 2LTH_{KV}d_h.$$

With b bytes per cached scalar:

$$\boxed{M_{\text{KV}} = 2LTH_{KV}d_hb.}$$

For a batch with unequal cached lengths $T_1, \ldots, T_B$, an idealized payload count is

$$M_{\text{KV}} = 2LH_{KV}d_hb\sum_{r=1}^{B} T_r.$$

Actual serving systems may reserve blocks, pad, page, quantize, offload, or maintain metadata, so allocated memory can differ from this tensor-payload calculation.

This formula is the clean bridge between architecture and systems behavior: MHA, GQA, and MQA change H_{KV} while often keeping H_Q fixed.

6. Decode bandwidth: why fewer KV heads help

A simple payload model for one layer at a decode step is proportional to

$$2TH_{KV}d_hb,$$

because the new query must interact with keys and values from the cached prefix. This is not a universal HBM-byte count: real kernels exploit cache hierarchy, tiling, batching, vectorization, and reuse. But it exposes the architectural lever.

If 32 query heads each own distinct K/V heads, the cache contains 32 K heads and 32 V heads per token. If the same 32 query heads share only 8 K/V heads, the persistent cache payload is one quarter as large. If all 32 queries share one K/V head, it is one thirty-second as large as the MHA cache under the same L, T, d_h, b assumptions.

The attention-score arithmetic is a different story. There are still 32 query heads producing 32 attention outputs. Sharing a K/V head does not erase those query-head computations. It primarily reduces K/V projection multiplicity, cache capacity, and the amount of distinct cached K/V data that needs to be supplied to attention.

> This is why "GQA reduces attention complexity by 4x" is usually too vague to be useful. Which complexity? K/V projection work and cache payload can fall by 4x in a 32Q/8KV example, while dense query-key scoring still has 32 query heads.

7. Multi-Query Attention (MQA)

Multi-Query Attention keeps multiple query heads but shares one key head and one value head across all of them. Shazeer introduced the variant specifically to reduce the K/V tensor size and memory-bandwidth requirements of incremental decoding.[2]

The defining counts are

$$H_Q = H, \qquad H_{KV} = 1.$$

Each query head still has its own query projection and therefore can produce a different attention distribution. What is shared is the representation of the prefix on the key and value side.

For query head i,

$$\boldsymbol{o}_{t,i} = \sum_{j \le t} \mathrm{softmax}_j\left(\frac{\boldsymbol{q}_{t,i}^\top \boldsymbol{k}_j}{\sqrt{d_h}}\right) \boldsymbol{v}_j.$$

The same $\boldsymbol{k}_j$ and $\boldsymbol{v}_j$ are read by every query head.

> **MQA does not mean one attention head.** It means one *KV head pair*. The model can still have many independent query heads and many distinct attention distributions.

That distinction is one of the most common implementation and explanation errors around MQA.

8. What MQA saves - and what it does not

Under the MHA/MQA cache formula, replacing H KV heads with one changes

$$2LTHd_hb \quad \longrightarrow \quad 2LTd_hb.$$

The cache reduction factor is H if all other terms remain equal.

MQA also shrinks the K and V projection output widths and usually their parameter counts. However:

- the number of query heads need not change;
- the output still contains one result per query head before the output projection;
- dense attention still compares every query head with the attended token positions;
- sequence-length dependence is not magically removed.

The original MQA paper reported much faster decoding in its experimental setting with only minor quality degradation relative to its baseline.[2] That empirical result should not be promoted into a theorem that MQA is always quality-neutral. Later work, including the GQA and DeepSeek-V2 studies, provides examples where more aggressive KV sharing loses quality relative to MHA in their tested settings.[3][4]

The architecture therefore exposes a real trade-off: extreme K/V sharing can be exceptionally cache-efficient, but it also reduces the number of distinct learned K/V subspaces available to the query heads.

9. Grouped-Query Attention (GQA)

GQA inserts intermediate points between MHA and MQA. Instead of H KV heads or one KV head, it uses some intermediate number G of KV heads. Query heads are divided into groups, and all queries in one group share that group's key and value head.[3]

Let

$$H_Q = H, \qquad H_{KV} = G, \qquad 1 < G < H,$$

and assume H is divisible by G. The number of query heads per KV group is

$$r = \frac{H}{G}.$$

A simple group map is

$$g(i) = 1 + \left\lfloor \frac{i-1}{r} \right\rfloor,$$

so query head i attends using key/value head $g(i)$.

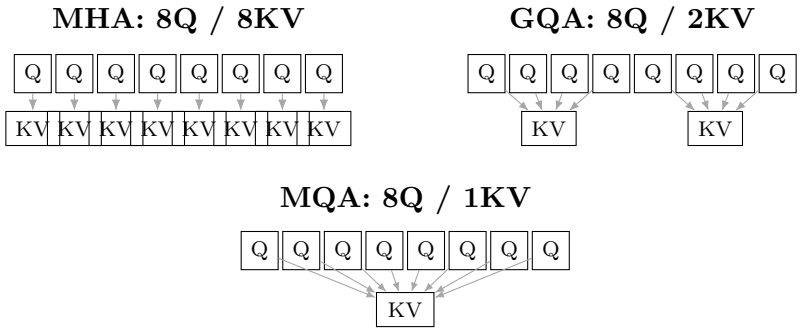
The output becomes

$$\boldsymbol{o}_{t,i} = \sum_{j \le t} \mathrm{softmax}_j \left(\frac{\boldsymbol{q}_{t,i}^{\top} \boldsymbol{k}_{j,g(i)}}{\sqrt{d_h}} \right) \boldsymbol{v}_{j,g(i)}.$$

This makes the continuum exact:

$$G = H \Rightarrow \text{MHA}, \qquad 1 < G < H \Rightarrow \text{GQA}, \qquad G = 1 \Rightarrow \text{MQA}.$$

10. One diagram for MHA, GQA, and MQA



The diagram intentionally draws a single "KV" box for a paired key/value head. The number of Q boxes is unchanged across all three examples. Only the number of distinct KV pairs changes.

For a fixed d_h, the cache payload is therefore proportional to 8, 2, and 1 respectively. But the number of query-head outputs remains eight in every case.

This picture is the simplest reliable way to read a modern model config: identify the query-head count and the KV-head count separately.

11. GQA was also an uptraining recipe

The GQA paper did more than name an intermediate architecture. It asked whether an existing MHA checkpoint could be converted into MQA or GQA without retraining from scratch.[3]

For conversion, K/V heads within each target group are mean-pooled to initialize the shared K/V head. The converted model is then continued in pretraining. The paper's headline experiments used only 5% of the original pretraining compute for this uptraining stage.[3]

This historical detail matters because two ideas are often collapsed:

- **GQA as an architecture:** fewer KV heads than query heads.
- **GQA uptraining:** a particular way to convert an MHA checkpoint into that architecture.

A model can be trained with GQA from scratch; it does not have to originate from an MHA checkpoint. Conversely, the mean-pooling initialization is part of the conversion recipe, not the mathematical definition of grouped-query attention.

Current Hugging Face Llama configuration documentation still describes this mean-pooling conversion convention for constructing grouped K/V heads from an MHA checkpoint.[7]

12. What the original GQA result actually showed

The GQA paper's most cited result came from T5-XXL models that were converted and uptrained for 5% of original pretraining compute. In that setup, the authors reported:[3]

Model	Time/sample (s)	Avg. score	Interpretation
MHA-XXL	1.51	47.2	quality reference
MQA-XXL	0.24	46.6	fastest of these three
GQA-8-XXL	0.28	47.1	near MQA time, near MHA average

The average combines the paper's summarization, translation, and question-answering evaluation set; the metrics differ by task. The table above is a deliberately reduced selection of the paper's reported results, not a new universal benchmark.

The correct takeaway is narrow: **in that experiment**, GQA-8 recovered most of the MHA baseline's average quality while retaining inference time close to MQA. The paper did not prove that every decoder-only LLM, kernel, batch size, or hardware platform will exhibit the same quality-speed curve.

That scoped reading is more useful than the slogan "GQA gives MHA quality at MQA speed."

13. A worked 32Q / 8KV grouping example

Suppose a layer has

$$H_Q = 32, \qquad H_{KV} = 8, \qquad d_h = 128.$$

Then each KV head serves

$$r = \frac{32}{8} = 4$$

query heads.

A natural grouping is:

$$\{Q_1, Q_2, Q_3, Q_4\} \to KV_1,$$
$$\{Q_5, Q_6, Q_7, Q_8\} \to KV_2,$$

and so on through KV_8.

For one token at one layer:

- query representation width: $32 \times 128 = 4096$ scalars;
- key cache width: $8 \times 128 = 1024$ scalars;
- value cache width: $8 \times 128 = 1024$ scalars;
- total cached K/V width: 2048 scalars.

Under MHA with 32 KV heads, the cached width would be 8192 scalars. GQA therefore reduces the raw K/V cache payload by 4x in this example while retaining 32 query heads.

That last phrase is the point: the model still has 32 query heads. GQA changes how many distinct K/V representations those queries consult.

14. Worked cache calculation: MHA vs GQA vs MQA

Use the same configuration across all variants:

$$L = 32, \quad T = 8192, \quad H_Q = 32, \quad d_h = 128, \quad b = 2 \text{ bytes}.$$

MHA: $H_{KV} = 32$

$$M = 2 \cdot 32 \cdot 8192 \cdot 32 \cdot 128 \cdot 2 = 4{,}294{,}967{,}296 \text{ bytes} = 4 \text{ GiB}.$$

GQA: $H_{KV} = 8$

$$M = 1{,}073{,}741{,}824 \text{ bytes} = 1 \text{ GiB}.$$

MQA: $H_{KV} = 1$

$$M = 134{,}217{,}728 \text{ bytes} = 128 \text{ MiB}.$$

Variant	KV heads	Cache	Relative to MHA
MHA	32	4 GiB	1x
GQA	8	1 GiB	1/4
MQA	1	128 MiB	1/32

These are tensor-payload numbers for one sequence. They exclude allocator metadata, fragmentation, other activations, model weights, temporary workspace, and framework-specific storage.

15. Head sharing changes more than memory capacity

Reducing H_{KV} affects several parts of the model:

K/V projection width. In a conventional implementation, fewer KV heads mean narrower K and V projection outputs. Their parameter count and projection work can therefore fall roughly in proportion to H_{KV} when head width is fixed.

Cache capacity. The persistent per-token cache scales linearly with H_{KV}.

Distinct cache payload read during decode. Fewer unique KV vectors can reduce the amount of K/V data that must be supplied from memory, especially when kernels reuse a shared KV head across multiple queries.

Representational sharing. Multiple query heads now interrogate the same K/V representation. This is an architectural constraint, not just a systems optimization.

What does not automatically fall by the same factor is the full dense attention arithmetic. Every query head still forms its own attention scores and weighted output. The exact runtime impact depends on the relative costs of projections, attention math, memory traffic, kernel design, batch size, and hardware.

16. Modern models made GQA mainstream

GQA moved quickly from a conversion technique into a standard model-design choice. Llama 2 used GQA in its larger 34B and 70B models for improved inference scalability.[8] Mistral 7B also explicitly adopted GQA for faster inference.[9] Meta then used GQA across both the 8B and 70B Llama 3 sizes.[10]

These examples matter less as a popularity contest than as evidence of the design pressure: long contexts, larger batches, and high-throughput serving make the cost of duplicating K/V state across every query head increasingly visible.

Still, "modern LLM = GQA" would be too broad. Architectures continue to explore other strategies, including latent cache compression such as MLA and additional forms of KV compression or eviction. The important lesson is that KV representation is now a first-class architectural decision.

> Model examples are implementation facts, not definitions. GQA remains GQA regardless of which named models adopt it, and model families can change attention designs across sizes or generations.

17. MLA is a different branch of the design space

MQA and GQA ask:

> How many conventional K/V heads should be stored?

Multi-head Latent Attention asks a different question:

> Do we need to store the conventional per-head K/V tensors at all?

DeepSeek-V2 introduced MLA with low-rank *joint* compression of keys and values. Instead of caching a full key vector and full value vector for every head and token, the hidden state is projected into a compact latent vector from which the content K/V representations are derived.[4]

This is why MLA should not be placed on a simple one-dimensional axis labeled "number of KV heads." Its main compression parameter is the latent width d_c, not H_{KV}.

The design also has a separate small positional key component because ordinary RoPE interferes with the algebraic projection absorption that makes latent caching efficient. That interaction is central to understanding MLA rather than an implementation footnote.

18. Low-rank joint K/V compression

Let $\boldsymbol{h}_t \in \mathbb{R}^{d_{\text{model}}}$ be the layer input for token t. MLA first compresses it into a shared K/V latent

$$\boldsymbol{c}_t^{KV} = W_D^{KV} \boldsymbol{h}_t, \qquad \boldsymbol{c}_t^{KV} \in \mathbb{R}^{d_c}.$$

Content keys and values can then be generated from that latent:

$$\boldsymbol{k}_t^C = W_U^K \boldsymbol{c}_t^{KV}, \qquad \boldsymbol{v}_t^C = W_U^V \boldsymbol{c}_t^{KV}.$$

DeepSeek-V2 uses d_c much smaller than the full concatenated multi-head K or V width.[4]

The phrase *low-rank* should be interpreted relative to the large full K/V projection space. The latent does not have to be smaller than one individual head dimension. In DeepSeek-V2, for example, $d_h = 128$ while $d_c = 512$. What matters is that 512 is far below the full 128-head content width $128 \times 128 = 16{,}384$.[4]

During optimized inference, the latent $\boldsymbol{c}_t^{KV}$ is the reusable content state. That is the structural break from ordinary MHA/GQA/MQA caching.

19. The absorption trick: do not reconstruct full K/V just to cache them

At first glance, latent compression seems to save storage only by adding decompression work: reconstruct $\boldsymbol{k}^C$ and $\boldsymbol{v}^C$ from $\boldsymbol{c}^{KV}$ every time they are needed. MLA avoids that naive implementation through associativity of matrix multiplication.[4]

For one head, the content score contains

$$(\boldsymbol{q}_i^C)^\top (W_{U,i}^K \boldsymbol{c}_j^{KV}).$$

Reassociate it as

$$\left((W_{U,i}^K)^\top \boldsymbol{q}_i^C\right)^\top \boldsymbol{c}_j^{KV}.$$

The key up-projection is effectively moved to the query side. Cached prefix tokens can remain in latent space.

For values,

$$\sum_j p_{ij} (W_{U,i}^V \boldsymbol{c}_j^{KV}) = W_{U,i}^V \left(\sum_j p_{ij} \boldsymbol{c}_j^{KV} \right).$$

The value up-projection can be applied after the weighted latent aggregation and combined with surrounding output-side linear maps.

> The practical idea is not "compress, then fully decompress every cached token." It is to rewrite the surrounding linear algebra so attention can operate with the compressed latent while fixed projection matrices are absorbed into adjacent computation.

20. Query compression is a separate idea

DeepSeek-V2 also applies a low-rank projection on the query path:

$$\boldsymbol{c}_t^Q = W_D^Q \boldsymbol{h}_t, \qquad \boldsymbol{q}_t^C = W_U^Q \boldsymbol{c}_t^Q.$$

The paper's stated motivation is to reduce activation memory during training. It explicitly notes that query compression does *not* reduce the inference KV cache.[4]

That makes intuitive sense. Queries are produced for the current token and are not the persistent history that future decode steps need to retain. Compressing the query pathway can change training activation storage and parameterization, but the long-lived serving cache is on the key/value side.

This distinction is useful beyond MLA: when evaluating any "attention compression" claim, ask *what is being compressed and when does that tensor live?* A smaller temporary query activation and a smaller persistent KV cache solve different systems problems.

21. Why ordinary RoPE breaks simple MLA absorption

RoPE applies a position-dependent rotation to queries and keys. In ordinary attention that is exactly what we want: query-key compatibility depends on relative position through the rotations. But a position-dependent operator inserted between linear projections changes the algebra available to MLA.[5]

Without positional rotation, a content score can be reassociated so the fixed key up-projection moves to the query side. With RoPE on the reconstructed content key, the score schematically contains a position-dependent transform between the query-side vector and the key up-projection:

$$\boldsymbol{q}_t^\top R_t^\top R_j W_U^K \boldsymbol{c}_j^{KV}.$$

Because R_j depends on the cached token position j, there is no single position-independent matrix that can simply absorb W_U^K for all prefix positions.

DeepSeek-V2 describes this as an incompatibility between standard RoPE placement and the desired low-rank KV absorption. Reconstructing position-specific full keys for the entire prefix at every decode step would defeat much of the inference benefit.[4]

The solution is to keep positional information on a separate, smaller path.

22. Decoupled RoPE

MLA separates the key/query representation into a content part and a RoPE-carrying part. For query head i:

$$\boldsymbol{q}_{t,i} = [\boldsymbol{q}^C_{t,i}; \boldsymbol{q}^R_{t,i}],$$

while the key is

$$\boldsymbol{k}_{j,i} = [\boldsymbol{k}^C_{j,i}; \boldsymbol{k}^R_{j}].$$

The RoPE query component $\boldsymbol{q}^R_{t,i}$ is head-specific, but the RoPE key component $\boldsymbol{k}^R_j$ is shared across heads in the DeepSeek-V2 construction.[4]

The attention score therefore combines content and positional compatibility:

$$\frac{(\boldsymbol{q}^C_{t,i})^\top \boldsymbol{k}^C_{j,i} + (\boldsymbol{q}^R_{t,i})^\top \boldsymbol{k}^R_j}{\sqrt{d_h + d^R_h}}.$$

Only the small shared RoPE key must be added to the persistent cache alongside the K/V latent. The large content keys can remain implicit through projection absorption.

This design is a clean example of architectural co-design: the positional encoding is modified not because RoPE is "bad," but because its usual placement blocks an inference optimization the architecture wants to exploit.

23. What MLA actually caches

With decoupled RoPE, DeepSeek-V2 caches two pieces per token per layer:

1. the joint K/V latent $\boldsymbol{c}_t^{KV} \in \mathbb{R}^{d_c}$;
2. the shared positional key $\boldsymbol{k}_t^R \in \mathbb{R}^{d_h^R}$.

Therefore the cache payload is

$$N_{\text{MLA}} = LT(d_c + d_h^R)$$

scalars, or

$$\boxed{M_{\text{MLA}} = LT(d_c + d_h^R)b.}$$

Notice what disappeared relative to ordinary K/V caching:

- no leading factor of 2 for separately cached full K and V tensors;
- no H_{KV} term describing a collection of conventional KV heads.

That is why forcing MLA into the standard formula $2LTH_{KV}d_hb$ is conceptually wrong. The cache object is different.

For variable-length batches, the idealized payload generalizes to

$$M_{\text{MLA}} = L(d_c + d_h^R)b\sum_r T_r.$$

Serving allocators and kernels add their own implementation details on top of this architectural payload.

24. DeepSeek-V2's concrete MLA dimensions

DeepSeek-V2 uses 60 Transformer layers, hidden dimension 5120, 128 attention heads, content head width $d_h = 128$, KV latent width $d_c = 512$, query latent width 1536, and decoupled RoPE width $d_h^R = 64$.[4]

The important ratios are

$$d_c = 4d_h, \qquad d_h^R = \frac{d_h}{2}.$$

Therefore the cached payload per token per layer is

$$d_c + d_h^R = 4.5d_h = 576 \text{ scalars.}$$

Across 60 layers:

$$60 \times 576 = 34{,}560$$

scalars per token. That matches the 34.6K-element figure reported for the large MLA model in the paper's attention ablation table.[4]

A subtle but valuable architecture lesson appears here: DeepSeek-V2's $d_h = 128$ and 128 heads do not imply a 16,384-dimensional residual stream. The model hidden dimension is 5120. Head width and hidden width are architectural parameters, not identities that must satisfy $d_{\text{model}} = Hd_h$ in every design.

25. Why the paper says MLA is like "GQA with 2.25 groups"

For ordinary GQA, the per-token per-layer cache contains

$$2Gd_h$$

scalars, where G is the number of KV groups/heads. For DeepSeek-V2 MLA, it contains

$$d_c + d_h^R = 4.5d_h.$$

Equating payload sizes:

$$2Gd_h = 4.5d_h \quad \Rightarrow \quad G = 2.25.$$

This is a **cache-size equivalence**, not a literal architecture with 2.25 KV heads. Head counts are discrete. The comparison says that DeepSeek-V2's MLA cache stores as many elements as a hypothetical conventional GQA cache with 2.25 groups.[4]

The comparison is useful precisely because it shows why MLA is not simply another head-sharing point. Its cache can occupy a size that does not correspond to any integer conventional KV-head count, because its stored object is a latent plus a positional component.

> Do not turn "equivalent to GQA with 2.25 groups" into "MLA has 2.25 KV heads." It does not.

26. An illustrative MHA/GQA/MQA/MLA cache comparison

Return to the earlier educational setup:

$$L = 32, \quad T = 8192, \quad H_Q = 32, \quad d_h = 128, \quad b = 2.$$

For an MLA-sized example, use DeepSeek-V2's ratios $d_c = 512$ and $d_h^R = 64$. Then

$$M_{\text{MLA}} = 32 \cdot 8192 \cdot 576 \cdot 2 = 301{,}989{,}888 \text{ bytes} = 288 \text{ MiB}.$$

Variant	Cache	vs. MHA	What is cached
MHA, 32KV	4 GiB	1x	32 K + 32 V heads
GQA, 8KV	1 GiB	1/4	8 K + 8 V heads
MQA, 1KV	128 MiB	1/32	1 K + 1 V head
MLA-sized example	288 MiB	0.0703x	512-d latent + 64-d RoPE key

A useful surprise is that MQA is smaller than this MLA parameterization in raw cache capacity. MLA's purpose is not "always produce the smallest cache." It aims for a richer learned attention mechanism while still compressing the cache aggressively.

That is a much more accurate mental model than ranking the four methods by memory alone.

27. The 93.3% DeepSeek-V2 number, correctly scoped

DeepSeek-V2's abstract reports a 93.3% reduction in KV cache compared with **DeepSeek 67B**, its earlier model.[4]

That number is frequently repeated as if "MLA reduces KV cache by 93.3%." The generalization is not justified. MLA cache reduction depends on the baseline architecture and on values such as H_{KV}, d_h, d_c, d_h^R, layer count, and storage precision.

There are at least three different comparisons one might make:

- DeepSeek-V2 vs. DeepSeek 67B as deployed in the paper;
- MLA vs. a hypothetical same-shape MHA architecture;
- MLA vs. a GQA or MQA architecture with a chosen number of KV heads.

They produce different percentages. The paper's 93.3% figure belongs only to the first comparison.

> **Publication rule.** Never quote a compression percentage without naming the baseline. "93.3%" is a DeepSeek-V2-vs-DeepSeek-67B result, not a constant of MLA.

28. What DeepSeek's ablations do and do not establish

DeepSeek-V2 includes a controlled 7B dense comparison of MQA, GQA with 8 groups, and MHA. On the four reported hard benchmarks, MHA scored highest, GQA-8 was intermediate, and MQA was lowest.[4]

A small selection illustrates the pattern:

Benchmark	MQA	GQA-8	MHA
BBH (3-shot EM)	33.2	35.6	37.0
MMLU (5-shot acc.)	37.9	41.2	45.2

The same paper separately compares MLA and MHA in matched small and large MoE setups. MLA uses far smaller reported KV caches and is higher on most listed benchmark values; the small MLA model is slightly lower on C-Eval while higher on BBH, MMLU, and CMMLU, and the large MLA model is higher on all four listed benchmarks.[4]

The careful interpretation is empirical: **for those architectures, data, training recipes, and evaluations**, DeepSeek found MLA to be a stronger cache-quality trade-off than the compared alternatives. It is not a theorem that MLA universally outperforms MHA or GQA.

29. The four designs side by side

	MHA	GQA	MQA	MLA
Query heads	many	many	many	many
Conventional KV heads cached	one per Q head	fewer than Q heads	one shared pair	latent is the main cache object
Primary cache lever	none	KV-head sharing	one shared KV pair	low-rank joint KV latent
Per-token/layer cache	$2H_Q d_h$	$2H_{KV} d_h$	$2d_h$	$d_c + d_h^R$
Persistent positional piece	ordinary K includes it	ordinary K includes it	ordinary K includes it	shared RoPE key in DeepSeek-V2
Main motivation	independent K/V per query head	quality/efficiency middle ground	aggressive KV sharing	compressed cache with richer capacity

The table highlights two independent ideas:

Head sharing controls how many conventional K/V subspaces exist.

Latent compression controls what representation is stored at all.

Future attention mechanisms can mix or extend these axes, so the most durable skill is to read the actual tensor shapes and cache contents rather than memorize four acronyms.

30. FlashAttention is a separate execution axis

FlashAttention changes the execution schedule of dense attention to reduce expensive memory traffic for attention intermediates while computing exact dense softmax attention. It does not define how many KV heads a model has.[11]

MHA/MQA/GQA/MLA are architectural choices about Q/K/V representation and, especially here, what K/V state is produced and cached. A model can therefore combine GQA with a FlashAttention-family kernel. Current PyTorch documentation explicitly supports GQA in several scaled-dot-product-attention backends, including FlashAttention on CUDA under documented constraints.[6]

The two questions are different:

- **Attention architecture:** what Q/K/V tensors exist?
- **Attention kernel:** how is the mathematical operation scheduled on hardware?

This is the same reason "FlashAttention reduces the KV cache" is misleading. FlashAttention primarily avoids materializing quadratic attention intermediates; MQA/GQA/MLA attack the persistent key/value representation.

31. PagedAttention is another different axis

PagedAttention was introduced to address memory management of the KV cache in LLM serving: fragmentation, allocation, sharing, and flexible placement of cache blocks.[12]

It does not prescribe whether a model uses MHA, GQA, or MQA. Those choices determine the logical KV payload. Paging determines how that payload is managed in a serving memory system.

MLA makes the distinction even clearer because its persistent cache object is not ordinary full K/V. A serving engine can still need block allocation, scheduling, and memory management for MLA state, but the representation being managed is architecture-specific.

A useful three-axis model is therefore:

1. **Representation:** MHA / GQA / MQA / MLA.
2. **Kernel execution:** standard attention / FlashAttention-family kernels / other optimized kernels.
3. **Serving memory management:** contiguous allocation / paged cache / offload / quantization / prefix reuse.

Optimizing one axis does not make the others disappear.

32. Current PyTorch GQA behavior

As of September 2026, `torch.nn.functional.scaled_dot_product_attention` exposes an `enable_gqa` option and documents GQA as an experimental feature.[6]

The documented shape notation distinguishes H_q, the number of query heads, from H, the number of key/value heads. Current constraints include

$$H_q \bmod H = 0, \qquad H_K = H_V.$$

The documentation includes a reference-equivalent Python implementation that expands key/value heads with `repeat_interleave`. It does so to match query-head count. The snippet describes equivalent semantics; it should not be read as a claim that every optimized backend physically materializes repeated K/V tensors.

Current documentation also notes backend-specific support and limitations, which can change faster than the architectural mathematics. Therefore production code should treat installed framework and kernel documentation as authoritative for device, dtype, mask, nested-tensor, and GQA support.

> Framework support is version-specific. The mathematical definition of GQA is stable; which fused kernel accepts a given tensor shape is not.

33. How to read a model configuration

Many Transformer configs expose two separate fields:

```
num_attention_heads = Hq
num_key_value_heads = Hkv
```

Current Hugging Face Llama documentation states the usual interpretation:[7]

- `Hkv == Hq` $\Rightarrow$ MHA;
- `Hkv == 1` $\Rightarrow$ MQA;
- $1 <$ `Hkv` $<$ `Hq` $\Rightarrow$ GQA.

Suppose a decoder has 48 layers, 64 query heads, 8 KV heads, head width 128, and BF16 cache elements. For a 16,384-token single sequence,

$$M_{\mathrm{KV}} = 2 \cdot 48 \cdot 16384 \cdot 8 \cdot 128 \cdot 2 = 3 \text{ GiB}.$$

The architecture is GQA with eight query heads per KV head.

Do not apply this config-reading rule blindly to MLA. MLA needs architecture-specific fields such as KV latent rank and decoupled RoPE dimension because conventional `num_key_value_heads` no longer describes the cache.

34. Common mistakes

"MQA has one query head." False. MQA has one K/V head pair shared across multiple query heads.

"GQA reduces the number of attention heads." Usually misleading. It reduces KV-head multiplicity while retaining the query-head count.

"GQA makes dense attention arithmetic linear." False. It changes K/V representation and can reduce projection/cache costs; it does not remove dense token interactions.

"MLA is just more aggressive GQA." False. MLA's defining cache is a low-rank joint K/V latent plus a small positional key in the DeepSeek design.

"MLA always uses less cache than MQA." False. With DeepSeek-V2's $4.5d_h$ cache payload, equal-width MQA stores $2d_h$ per layer/token. MLA is pursuing a different quality-capacity trade-off.

"DeepSeek's 93.3% is the MLA compression ratio." False. It is the reported DeepSeek-V2 vs. DeepSeek 67B comparison.

"FlashAttention, GQA, and PagedAttention are competing versions of the same optimization." False. They operate on different axes: execution schedule, representation, and cache management.

"Head dimension must equal model width divided by head count." Common, but not universal.

35. A practical decision framework

When reading or designing an attention architecture, ask these questions in order:

1. **How many query heads exist?** This tells you how many distinct query-side attention channels are produced.
2. **How many conventional KV heads exist?** If this is lower than query-head count, the model uses KV sharing in the MQA/GQA family.
3. **What exactly is persisted during decode?** Full K/V tensors, a latent, a quantized state, selected tokens, or something else?
4. **How is position represented?** RoPE or another positional mechanism can affect what projection rearrangements are legal.
5. **Which costs actually fall?** Projection FLOPs, cache capacity, cache traffic, attention arithmetic, allocator overhead, or wall-clock latency are different metrics.
6. **What evidence supports the quality claim?** Training setup and benchmark matter.

This framework is more durable than memorizing a linear evolution from MHA to MQA to GQA to MLA. Architectures do not evolve on one axis. They trade representation capacity, memory footprint, bandwidth, arithmetic, and implementation complexity in different combinations.

> **The compact mental model:** MHA gives every query head its own K/V pair. MQA shares one pair across all queries. GQA shares one pair within each query group. MLA changes the stored object from full per-head K/V to a learned latent representation plus a small positional component.

36. Connections to the rest of the Understanding AI series

This topic sits at the intersection of four previous handbooks:

Attention. MHA is the baseline multi-head mechanism; MQA/GQA/MLA preserve dense attention while changing how query/key/value representations are organized.

KV Cache. The cache formula makes the systems consequences of KV-head multiplicity explicit. MLA then demonstrates why the ordinary cache formula is architecture-specific rather than universal.

RoPE. MLA's decoupled RoPE exists because position-dependent rotations interfere with the projection absorption used for latent K/V caching.

FlashAttention. FlashAttention optimizes how attention runs; MQA/GQA/MLA change what attention tensors exist and what state is retained.

Together, these concepts form a useful hierarchy:

$$\text{mathematical attention} \to \text{head architecture} \to \text{positional encoding} \to \text{persistent cache} \to \text{kernel execution}.$$

Keeping those layers separate is one of the best ways to reason accurately about modern Transformer inference.

References and further reading

[**1**] Vaswani, A. et al. *Attention Is All You Need.* NeurIPS 2017. `https://arxiv.org/abs/1706.03762`

[**2**] Shazeer, N. *Fast Transformer Decoding: One Write-Head Is All You Need.* 2019. `https://arxiv.org/abs/1911.02150`

[**3**] Ainslie, J. et al. *GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints.* EMNLP 2023. `https://aclanthology.org/2023.emnlp-main.298/`

[**4**] DeepSeek-AI et al. *DeepSeek-V2: A Strong, Economical, and Efficient Mixture-of-Experts Language Model.* 2024. `https://arxiv.org/abs/2405.04434`

[**5**] Su, J. et al. *RoFormer: Enhanced Transformer with Rotary Position Embedding.* Neurocomputing 2024; preprint first posted 2021. `https://arxiv.org/abs/2104.09864`

[**6**] PyTorch. *torch.nn.functional.scaled_dot_product_attention* documentation. Current behavior checked September 2026. `https://docs.pytorch.org/docs/stable/generated/torch.nn.functional.scaled_dot_product_attention`

[**7**] Hugging Face Transformers. *Llama configuration documentation*, including `num_attention_heads` and `num_key_value_heads`. Current documentation checked September 2026. `https://huggingface.co/docs/transformers/main/model_doc/llama`

[**8**] Touvron, H. et al. *Llama 2: Open Foundation and Fine-Tuned Chat Models.* 2023. `https://arxiv.org/abs/2307.09288`

[**9**] Jiang, A. Q. et al. *Mistral 7B.* 2023. `https://arxiv.org/abs/2310.06825`

[**10**] Meta AI. *Introducing Meta Llama 3.* 2024. `https://ai.meta.com/blog/meta-llama-3/`

[**11**] Dao, T. et al. *FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness.* NeurIPS 2022. `https://arxiv.org/abs/2205.14135`

[**12**] Kwon, W. et al. *Efficient Memory Management for Large Language Model Serving with PagedAttention.* SOSP 2023. `https://arxiv.org/abs/2309.06180`

Source discipline. Paper-specific empirical results in this handbook are presented as results of the cited experiments, not universal guarantees. Framework behavior is explicitly time-scoped where it can change.

Reference map: what to read for which question

Question	Best starting source
Where did multi-head attention come from?	Vaswani et al. [1]
Why was MQA proposed?	Shazeer [2]
How is GQA defined, and how was MHA converted to it?	Ainslie et al. [3]
What exactly does MLA cache?	DeepSeek-V2 [4], Section 2.1
Why does MLA decouple RoPE?	DeepSeek-V2 [4], Section 2.1.3
Where do the 2.25-group and 93.3% numbers come from?	DeepSeek-V2 [4]
What did the controlled MQA/GQA/MHA and MLA/MHA ablations report?	DeepSeek-V2 [4], Appendix D
How does current PyTorch expose GQA?	PyTorch SDPA docs [6]
How do current Llama configs encode KV-head count?	Hugging Face Llama docs [7]
Which Llama 2 sizes used GQA?	Llama 2 report [8]
Where is GQA used in Mistral 7B?	Mistral 7B [9]
How are FlashAttention and PagedAttention different from head architecture?	FlashAttention [11], PagedAttention [12]

Implementation support changes faster than the mathematics. For deployed systems, treat the exact installed framework, model code, and attention-kernel documentation as authoritative for supported devices, dtypes, masks, head dimensions, cache layouts, and fused GQA/MLA behavior.